

Application mobile lecture de QR codes

Éléments gratuits nécessaires : API, base de données, serveur...

Projet GestionInvitations

1. Objectif

Créer une **application mobile dédiée** (Android / iOS) pour scanner les QR codes des invitations générées par GestionInvitations, valider l'entrée des invités en temps réel, et synchroniser le statut (scanné / déjà entré) avec une base centrale.

*Aujourd'hui, le scanner est intégré au PC (OpenCV + SQLite locale). L'app mobile nécessite un **backend en ligne** : le téléphone ne peut pas lire directement le fichier SQLite du PC.*

2. Architecture cible (vue d'ensemble)

1. **App PC GestionInvitations** — crée les événements, invités, génère les QR (format actuel : INVITE-`{id}`-`{hash}`).
2. **API / serveur** — reçoit les données d'invités et répond aux scans mobiles.
3. **Base de données cloud** — stocke invités, QR, statut de scan, historique.
4. **App mobile** — caméra + lecture QR + appel API + affichage OK / Refusé.
5. **(Optionnel)** Tableau de bord web pour suivre les entrées en live.

3. Ce dont vous avez besoin (checklist)

Application mobile	Scanner QR + UI agent d'accueil	Oui	Flutter / React Native / Expo
API REST (backend)	Valider un QR, marquer l'entrée	Oui	FastAPI / Flask / Node sur Render ou Railway
Base de données	Invités, QR, scans, événements	Oui	PostgreSQL (Neon / Supabase) ou SQLite hébergé
Hébergement serveur	Faire tourner l'API 24/7 le jour J	Oui	Render Free, Railway Free, Fly.io Free
Authentification agents	Empêcher n'importe qui de valider	Fortement recommandé	JWT + login simple / Firebase Auth
Stockage fichiers	Photos / logs (optionnel)	Non	Cloudflare R2 free tier / local
Nom de domaine	URL propre pour l'API	Non	Sous-domaine gratuit *.onrender.com
Push notifications	Alerte double entrée	Non	Firebase Cloud Messaging (gratuit)
CDN / HTTPS	Sécuriser les appels API	Inclus	Fourni par l'hébergeur
Compte développeur stores	Publier sur Play Store / App Store	Si publication	Play : 25 \$ une fois / Apple : 99 \$/an

Note : pour un usage interne (APK installé sur les téléphones du staff), vous n'avez pas besoin de publier sur les stores.

4. Stack gratuite recommandée (simple et réaliste)

Proposition adaptée à votre projet Python existant — coût = **0 €** dans les limites des free tiers.

App mobile	Flutter (Dart) ou Expo (React Native)	Caméra QR mature, 1 code → Android+iOS	Illimité (outil open source)
Lecture QR	mobile_scanner (Flutter) / expo-camera	Gratuit, open source	—
API	FastAPI (Python)	Même langage que GestionInvitations	Selon hébergeur
Base	PostgreSQL via Supabase ou Neon	Gratuit, backup, SQL familier	~500 Mo / projets limités
Hébergement API	Render.com (Web Service Free)	HTTPS, déploiement Git	Veille après inactivité (~15 min)
Auth staff	Token JWT + PIN événement	Simple, sans service payant	—
Sync PC → cloud	Export / sync depuis l'app desktop	Réutilise votre SQLite actuelle	—

5. Alternative « ultra-simple » : Firebase / Supabase seuls

Si vous voulez éviter de coder un serveur API dédié :

- **Supabase** (gratuit) : base PostgreSQL + API auto-générée + auth.
- **Firebase** (gratuit) : Firestore + Auth + règles de sécurité.

L'app mobile lit/écrit directement dans la base via le SDK. L'app PC exporte les invités (JSON/CSV) vers Supabase/Firebase. Moins de contrôle métier, mais plus rapide à lancer.

6. API minimale à prévoir (endpoints)

Que vous codiez FastAPI ou utilisiez Supabase, voici les opérations indispensables :

POST	Connexion agent	identifiant + PIN événement	token JWT
GET	Infos événement	event_id	nom, date, nb invités
POST	Scanner / valider QR	qr_code, lieu, agent_id	valide + infos invité + message
GET	Statistiques live	event_id	entrés / restants / refus
POST	Sync invités (PC)	liste invités + QR	accusé de réception
GET	Historique scans	event_id	liste horodatée

6.1 Réponses typiques du scan

- **OK** — premier scan : afficher nom, titre, table, accompagnants ; marquer *statut* = *scanné*.
- **Déjà entré** — QR déjà validé : afficher heure du 1er scan (anti-fraude).
- **Invalide** — QR inconnu ou mauvais événement.
- **Événement fermé** — hors créneau autorisé (optionnel).

7. Base de données — tables essentielles

evenements	id, nom, date, lieu, pin_agent, actif	Correspond à votre modèle actuel
invites	id, evenement_id, titre, nom, table, qr_code UNIQUE, statut, date_scan	Migrer depuis SQLite PC
scans	id, invite_id, agent_id, lieu, horodatage, resultat	Historique (audit)
agents	id, nom, login, hash_mdp, role	Staff entrée / contrôle

Votre format QR actuel INVITE- {id} - {hash} peut être conservé : il suffit de synchroniser la colonne qr_code vers le cloud.

8. Serveur / hébergement — options 100 % gratuites

Render	Web Service Free	Simple, HTTPS, Git deploy	S'endort après ~15 min d'inactivité
Railway	Crédit mensuel gratuit	Rapide à lancer	Crédit limité / mois
Fly.io	Free allowance	Proche des utilisateurs	Carte parfois demandée
Supabase	Free tier	DB + API + auth inclus	Limite de projets / bande passante
Neon	Postgres serverless free	Excellente pour FastAPI	À coupler avec un hébergeur API
Cloudflare Tunnel	Gratuit	Exposer un PC local sans VPS	PC doit rester allumé le jour J

ngrok (free)	Tunnel temporaire	Tests rapides	URL change ; pas idéal en prod

Astuce jour J : si Render s'endort, faites un « ping » toutes les 10 min (cron gratuit UptimeRobot) ou gardez le PC allumé avec Cloudflare Tunnel + API locale.

9. Application mobile — éléments techniques

Écrans indispensables

- Connexion agent (PIN / mot de passe événement).
- Choix de l'événement actif.
- Scanner caméra plein écran + flash.
- Résultat : vert (OK) / orange (déjà entré) / rouge (invalide).
- Compteur live : entrés / total.
- Mode hors-ligne léger (option) : file d'attente des scans si réseau faible.

Permissions & matériel

- Permission caméra (obligatoire).
- Internet (WiFi / 4G sur site).
- Téléphones Android récents suffisent ; pas besoin de terminal payant.

10. Lien avec GestionInvitations (PC)

Sans modifier toute l'app desktop, le flux minimal est :

1. Générer les invitations + QR comme aujourd'hui (SQLite locale).
2. Ajouter un bouton « **Synchroniser vers le cloud** » qui envoie événement + invités + qr_code vers l'API.
3. Le jour J, seuls les téléphones scannent ; le PC peut afficher les stats live.
4. Après l'événement, optionnellement rapatrier les date_scan vers la base locale.

11. Sécurité (même en gratuit)

- HTTPS uniquement (fourni par Render / Supabase).
- Token JWT avec expiration courte pour les agents.
- PIN unique par événement (changeable).
- QR unique et non prévisible (votre hash actuel convient).
- Journal des scans (qui a validé, quand, où).
- Ne jamais embarquer la base complète en clair dans l'APK.

12. Budget réel : gratuit vs payant

Développement (Flutter / FastAPI)	Oui (open source)	Si vous engagez un développeur
API + DB (petits événements)	Oui (free tiers)	Si > quelques milliers d'invités / mois ou uptime critique
Distribution APK interne	Oui	—
Google Play	Non (25 \$ une fois)	Si publication publique
Apple App Store	Non (99 \$/an)	Si iPhone staff via store officiel
SMS / WhatsApp d'invitation	Hors scope QR	Services SMS payants

13. Plan de mise en œuvre suggéré

1	Créer projet Supabase ou Postgres + schéma tables	1–2 jours
2	API FastAPI : sync + validate QR	2–4 jours
3	App mobile Expo/Flutter : login + scan + résultat	1–2 semaines
4	Bouton sync dans GestionInvitations (PC)	1–2 jours
5	Test terrain (WiFi, double scan, hors ligne)	2–3 jours
6	APK pour staff + checklist jour J	1 jour

14. Recommandation finale

Pour démarrer **sans budget** et rester cohérent avec votre stack Python :

- **Mobile** : Flutter ou Expo
- **API** : FastAPI
- **Base** : Supabase (PostgreSQL) free
- **Hébergement API** : Render free (ou Tunnel Cloudflare le jour J)
- **Auth** : JWT + PIN événement
- **Sync** : bouton dans l'app PC existante

Vous n'avez pas besoin d'acheter un serveur, une licence de base de données, ni un SDK QR payant. Les seuls coûts éventuels sont la publication sur les stores et, plus tard, un hébergement payant si le free tier devient trop limité.

Document généré pour le projet GestionInvitations — usage interne.